Mikhail Razakov

C++ / Systems Engineer | LLVM · x86-64 · Algorithms

razakov.mikhail@outlook.com
t.me/Micodiy
github.com/Misha1302

C++/systems engineer with LLVM 22 internship experience and hands-on compiler work. I implement conservative IR transformations, native x86-64 codegen and memory-dependence analysis, then validate them with independent oracles and negative cases.

Professional and project evidence

MCST: LLVM 22/C++23 optimization · separate Roslyn/C# analyzer for an ISP RAS selection task.

x86-64 pipeline

SSA/CFG validation, liveness/ interference, linear scan, seeded simulated annealing, SysV codegen, and differential execution.

Conservative optimization

LLVM Global-IV: APInt affine evolution, transitive effects, explicit legality, and 29 verifier/idempotence/execution regressions.

EXPERIENCE

July 1 — August 31, 2026

MCST — Compiler Engineering Intern

- Implemented an LLVM LICM pass in C++; transformation legality is derived from loop structure, memory accesses, side effects, and speculative-safety constraints.
- Built a Python harness comparing execution before/after optimization and checking explicit transformation / no-transformation cases; also implemented RPO, Dijkstra, Dinic, and Tarjan SCC in C++23.

PROJECTS

LLVM Interprocedural Global IV Optimization

A conservative LLVM 22 prototype/MVP with an actual IR transform; 29 positive/negative regression cases cover verifier validity, idempotence, and before/after execution.

x86-64 Compiler Backend & Register Allocation

A reference interpreter and differential execution check generated-code semantics; the allocator assignment contract is verified independently.

PS-form Memory Dependence Analyzer

Conservative yes/no/maybe results checked by an independent exact oracle on small domains, randomized/metamorphic tests, and reproducible counterexamples.

SKILLS

C++ / toolchain

C++23, C17, LLVM 22, CMake, Linux, clang-tidy, ASan/UBSan

Algorithms / analysis

graph algorithms, CFG, dominance, SSA, SCC, dataflow, memory dependence

Machine-level systems

x86-64 SysV, code generation, liveness/interference, register allocation, generated-code analysis

Verification

exact/reference oracles, differential/metamorphic testing, isolated execution, reproducible counterexamples

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

HSE Faculty of Computer Science — open-source projects competition

Selected as one of the winners with UniversalToolchain / Wist2.

LangDev'26 — Build the Language, Then Make the Abstractions Disappear

Delivered the UniversalToolchain architecture talk at LangDev on 9 October 2026.

Moscow / remote